

EECS 182 Deep Neural Networks

Spring 2025 Anant Sahai

Homework 4

This homework is due on Sunday, March 2, 2024, at 10:59PM.

1. Convolutional Networks

Note: Throughout this problem, we will use the convention of NOT flipping the filter before dragging it over the signal. This is the standard notation with neural networks (ie, we assume the filter given to us is already flipped)

- List two reasons we typically prefer convolutional layers instead of fully connected layers when working with image data.**
- Consider the following 1D signal: $[1, 4, 0, -2, 3]$. After convolution with a length-3 filter, no padding, stride=1, we get the following sequence: $[-2, 2, 11]$. **What was the filter?**
(Hint: Just to help you check your work, the first entry in the filter that you should find is 2. However, if you try to use this hint directly to solve for the answer, you will not get credit since this hint only exists to help you check your work.)
- Transpose convolution is an operation to help us upsample a signal (increase the resolution). For example, if our original signal were $[a, b, c]$ and we perform transpose convolution with pad=0 and stride=2, with the filter $[x, y, z]$, the output would be $[ax, ay, az + bx, by, bz + cx, cy, cz]$. Notice that the entries of the input are multiplied by each of the entries of the filter. Overlaps are summed. Also notice how for a fixed filtersize and stride, the dimensions of the input and output are swapped compared to standard convolution. (For example, if we did standard convolution on a length-7 sequence with filtersize of 3 and stride=2, we would output a length-3 sequence).

If our 2D input is $\begin{bmatrix} -1 & 2 \\ 3 & 1 \end{bmatrix}$ and the 2D filter is $\begin{bmatrix} +1 & -1 \\ 0 & +1 \end{bmatrix}$ **What is the output of transpose convolution with pad=0 and stride=1?**

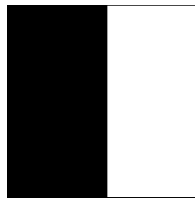
2. Convolutional Neural Networks

- Consider a convolutional neural network layer with:
 - Input shape: $[10, 3, 32, 32]$ (batch size, number of input channels, input height, input width)
 - Number of filters: 64

For each of the following configurations of kernel size, stride, and padding, **calculate the output dimensions** $[n, c, h, w]$ where n is the batch size, c is the number of channels, and h, w are the output height and width. Assume that each layer has the same input shape and number of filters as specified above.

- Kernel size: 3x3, stride: 1, padding: 1
- Kernel size: 4x4, stride: 2, padding: 0

- Design a 3x3 filter that detects vertical edges like the one shown in the image below.**



$$\begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

(c) **Design a 3x3 filter to blur an image.**

(Hint: blurring involves averaging a pixel's value with those of its neighbors.)

$$\begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

(d) In a regular convolutional operation, the kernel slides over the input data in a contiguous manner. However, in dilated convolution, the kernel is "dilated" by introducing gaps between its elements, resulting in a larger receptive field for each output pixel.

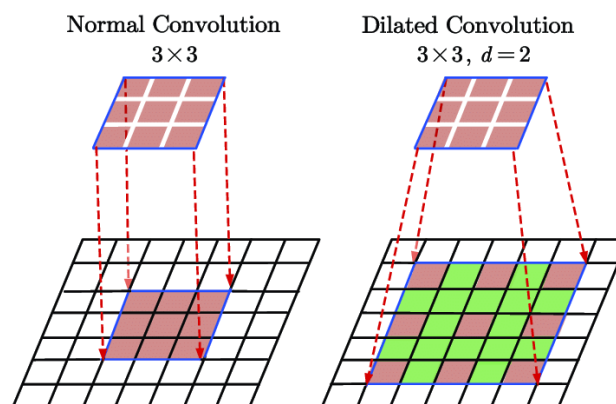


Figure 1: Dilated convolution kernels. Source: "Brain MRI Super-Resolution Using 3D Dilated Convolutional Encoder–Decoder Network" by Du et al.

The dilation (d) determines the spacing between kernel elements — a dilation of d introduces $d - 1$ gaps between two kernel elements. The example above illustrates a 3x3 1-dilated kernel ($d = 1$ is equivalent to a regular convolutional kernel) and a 3x3 2-dilated ($d = 2$) kernel.

- i. You are given an input matrix M and 2x2 filter k below. **Compute their dilated convolution with $d = 2$.** Assume that gaps in the kernel are filled with zeros, stride is 1 and padding is 0.

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad k = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

- ii. Consider a two-layer architecture:

$$\text{DilatedConv1}(3 \times 3, d=1) \rightarrow \text{DilatedConv2}(3 \times 3, d=2)$$

Both layers use the same sized kernel (3×3), but `DilatedConv1` has a dilation $d = 1$ and `DilatedConv2` has a dilation $d = 2$. **Compute the size of the receptive field at the output of the final layer (`DilatedConv2`).**

Recall that the receptive field is the region in the *original input* image whose pixels affect the output for a pixel in the specified layer.

3. Depthwise Separable Convolutions

Depthwise separable convolutions are a type of convolutional operation used in deep learning for image processing tasks. Unlike traditional convolutional operations, which perform both spatial and channel-wise convolutions simultaneously, depthwise separable convolutions decompose the convolution operation into two separate operations: Depthwise convolution and Pointwise convolution.

This can be viewed as a low-rank approximation to a traditional convolution. For simplicity, throughout this problem, we will ignore biases while counting learnable parameters.

- (a) Suppose the input is a three-channel 224×224 -resolution image, the kernel size of the convolutional layer is 3×3 , and the number of output channels is 4.

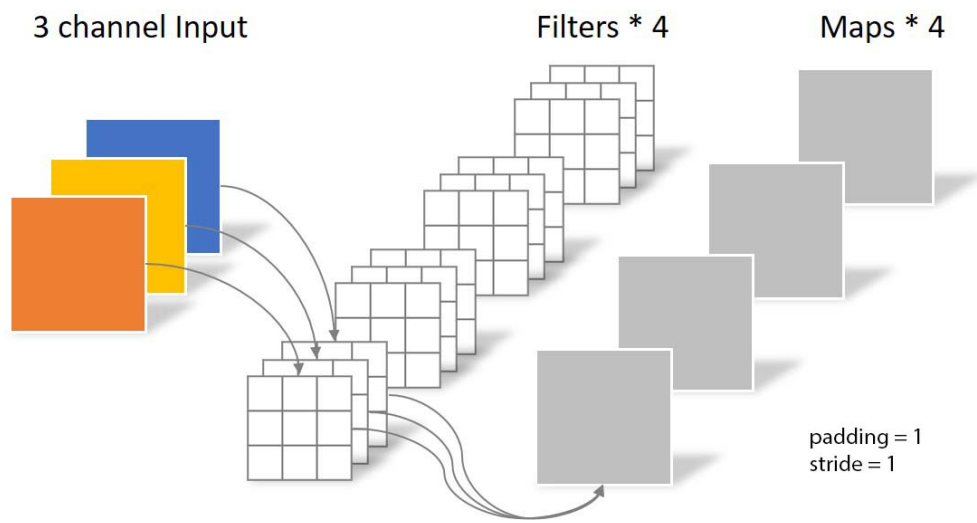


Figure 2: Traditional convolution.

What is the number of learnable parameters in the traditional convolution layer?

- (b) Depthwise separable convolution consists of two parts: depthwise convolutions (Fig.3) followed by pointwise convolutions (Fig.4). Suppose the input is still a three-channel 224×224 -resolution image. The input first goes through depthwise convolutions, where the number of output channels is the same as the number of input channels, and there is no “cross talk” between different channels. Then, this intermediate output goes through pointwise convolutions, which is basically a traditional convolution with the filter size being 1×1 . Assume that we have 4 output channels.

What is the total number of learnable parameters of the depthwise separable convolution layer which consists of both depthwise and pointwise convolutions?

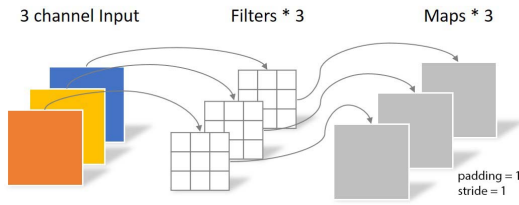


Figure 3: Depthwise convolution.

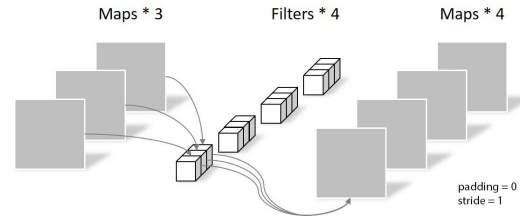


Figure 4: Pointwise convolution

4. Regularization and Dropout

Recall that linear regression optimizes the following learning objective:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 \quad (1)$$

One way of using *dropout* during SGD on the d -dimensional input features $\mathbf{x}_i$ involves *keeping* each feature at random $\sim_{i.i.d} \text{Bernoulli}(p)$ (and zeroing it out if not kept) and then performing a traditional SGD step.

It turns out that such dropout makes our learning objective effectively become

$$\mathcal{L}(\tilde{\mathbf{w}}) = \mathbb{E}_{R \sim \text{Bernoulli}(p)} \left[\|\mathbf{y} - (R \odot X)\tilde{\mathbf{w}}\|_2^2 \right] \quad (2)$$

where $\odot$ is the element-wise product and the random binary matrix $R \in \{0, 1\}^{n \times d}$ is such that $R_{i,j} \sim_{i.i.d} \text{Bernoulli}(p)$. We use $\tilde{\mathbf{w}}$ to remind you that this is learned by dropout.

Recalling how Tikhonov-regularized (generalized ridge-regression) least-squares problems involve solving:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \|\Gamma\mathbf{w}\|_2^2 \quad (3)$$

for some suitable matrix Γ , it turns out we can manipulate (2) to eliminate the expectations and get:

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - pX\tilde{\mathbf{w}}\|_2^2 + p(1-p)\|\tilde{\Gamma}\tilde{\mathbf{w}}\|_2^2 \quad (4)$$

with $\tilde{\Gamma}$ being a diagonal matrix whose j -th diagonal entry is the norm of the j -th column of the training matrix X .

(a) **Show that we can manipulate (2) to eliminate the expectations and get:**

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - pX\tilde{\mathbf{w}}\|_2^2 + p(1-p)\|\tilde{\Gamma}\tilde{\mathbf{w}}\|_2^2 \quad (5)$$

with $\tilde{\Gamma}$ being a diagonal matrix whose j -th diagonal entry is the norm of the j -th column of the training matrix X .

(b) **How should we transform the $\tilde{\mathbf{w}}$ we learn using (5) (i.e. with dropout) to get something that looks a solution to the traditionally regularized problem (3)?**

(Hint: This is related to how we adjust weights learned using dropout training for using them at inference time. PyTorch by default does this adjustment during training itself, but here, we are doing dropout slightly differently with no adjustments during training.)

(c) **With the understanding that the Γ in (3) is an invertible matrix, change variables in (3) to make the problem look like classical ridge regression:**

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - \tilde{X}\tilde{\mathbf{w}}\|_2^2 + \lambda\|\tilde{\mathbf{w}}\|_2^2 \quad (6)$$

Explicitly, what is the changed data matrix $\tilde{X}$ in terms of the original data matrix X and Γ ?

- (d) Continuing the previous part, with the further understanding that Γ is a *diagonal* invertible matrix with the j -th diagonal entry proportional to the norm of the j -th column in X , **what can you say about the norms of the columns of the effective training matrix $\tilde{X}$ and speculate briefly on the relationship between dropout and batch-normalization.**

5. Multiplicative Regularization beyond Dropout

In dropout, we get a regularizing effect by multiplying the activations of the previous layer by iid coin tosses to randomly zero out many of them during each SGD update. Here, we will consider a linear-regression problem but instead of randomly multiplying each input feature with a 0 or a 1 during SGD updates, we will multiply each feature of our input with an iid random sample of a normal distribution with mean μ and variance σ^2 . In other words, we perform the elementwise product $R \odot X$, where R is a matrix where every iid entry $R_{ij} \sim \mathcal{N}(\mu, \sigma^2)$ and $\odot$ represents elementwise multiplication.

It turns out that the expected training loss

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}_{R_{ij} \sim \mathcal{N}(\mu, \sigma^2)} \left[\|\mathbf{y} - (R \odot X)\mathbf{w}\|_2^2 \right]$$

can be put in the form

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - \text{---}(\mathbf{A})\text{---}X\mathbf{w}\|_2^2 + \text{---}(\mathbf{B})\text{---}\|\Gamma\mathbf{w}\|_2^2$$

where $\Gamma = (\text{diag}(X^\top X))^{1/2}$.

What are (A) and (B)?

Select one choice for **(A)**:

- ☐ μ
☐ 2μ
☐ $\frac{\mu}{2}$

- ☐ σ
☐ 2σ
☐ $\frac{\sigma}{2}$

Select one choice for **(B)**:

- ☐ μ^2
☐ $2\mu^2$
☐ $\frac{\mu^2}{2}$
- ☐ σ^2
☐ $2\sigma^2$
☐ $\frac{\sigma^2}{2}$

Show some work below to justify your choices. Correct answers with incorrect or no supporting work will not receive full credit.

(Hint: You might find it helpful to think about the case $\mu = 1$ and $\sigma^2 = 0$ to help you pick as well as $\mu = 0$ and $\sigma^2 = \infty$.)

6. Analyzing Distributed Training

For real-world models trained on lots of data, the training of neural networks is parallelized and accelerated by running workers on distributed resources, such as clusters of GPUs. In this question, we will explore three popular distributed training paradigms:

All-to-All Communication: Each worker maintains a copy of the model parameters (weights) and processes a subset of the training data. After each iteration, each worker communicates with every other worker and updates its local weights by averaging the gradients from all workers.

Parameter Server: A dedicated server, called the parameter server, stores the global model parameters. The workers compute gradients for a subset of the training data and send these gradients to the parameter

server. The server then updates the global model parameters and sends the updated weights back to the workers.

Ring All-Reduce: Arranges n workers in a logical ring and updates the model parameters by passing messages in a circular fashion. Each worker computes gradients for a subset of the training data, splits the gradients into n equally sized chunks and sends a chunk of the gradients to their neighbors in the ring. Each worker receives the gradient chunks from its neighbors, updates its local parameters, and passes the updated gradient chunks along the ring. After $n - 1$ passes, all gradient chunks have been aggregated across workers, and the aggregated chunks are passed along to all workers in the next $n - 1$ steps. This is illustrated in Fig. 5.

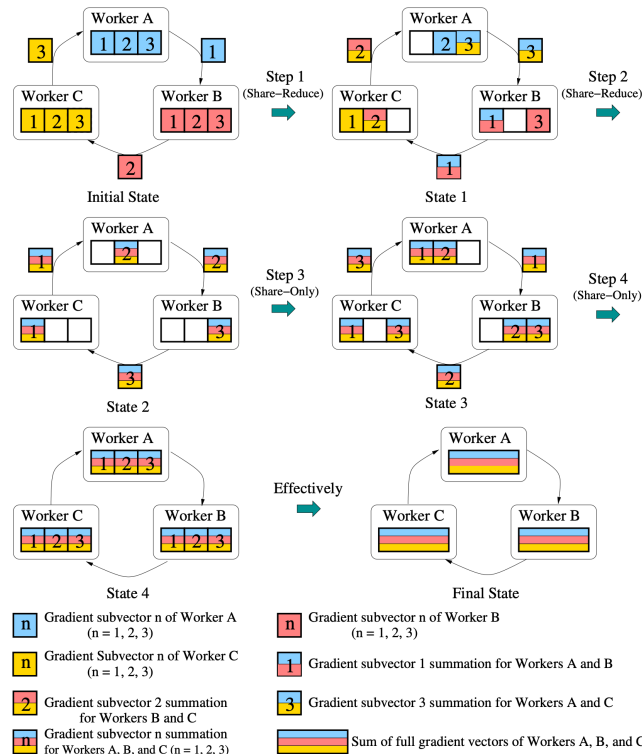


Figure 5: Example of Ring All-Reduce in a 3 worker setup. Source: Mu Et. al, *GADGET: Online Resource Optimization for Scheduling Ring-All-Reduce Learning Jobs*

	Number of Messages Sent	Size of each message
All-to-All		p
Parameter Server	$2n$	
Ring All-Reduce	$n(2(n - 1))$	

For each of the distributed training paradigms, fill in the total number of messages sent and the size of each message. Assume that there are n workers and the model has p parameters, with p divisible by n .

7. Memory considerations when using GPUs (Coding Question)

In this homework, you will run [GPUMemory.ipynb](#) to train a ResNet model on CIFAR-10 using PyTorch and explore its implications on GPU memory.

We will explore various systems considerations, such as the effect of batch size on memory usage and how different optimizers (SGD, SGD with momentum, Adam) vary in their memory requirements.

It is strongly recommended that you start early on this question, since colab daily GPU limits may require you to complete this question over a few days with breaks in between.

- (a) Managing GPU memory for training neural networks (Notebook Section 1).
 - (i) **How many trainable parameters does ResNet-152 have?** What is the **estimated size of the model in MB**?
 - (ii) Which GPU are you using? **How much total memory does it have?**
 - (iii) After you load the model into memory, **what is the memory overhead (MB) of the CUDA context loaded with the model?**
- (b) Optimizer memory usage (Notebook Section 2).
 - (i) **What is the total memory utilization during training with SGD, SGD with momentum and Adam optimizers?** Report in MB individually for each optimizer.
 - (ii) **Which optimizer consumes the most memory? Why?**
- (c) Batch size, learning rates and memory utilization (Notebook Section 3)
 - (i) **What is the memory utilization for different batch sizes (4, 16, 64, 256)? What is the largest batch size you were able to train?**
 - (ii) **Which batch size gave you the highest accuracy at the end of 10 epochs?**
 - (iii) **Which batch size completed 10 epochs the fastest (least wall clock time)? Why?**
 - (iv) Attach your **training accuracy vs wall time plots** with your written submission.

8. Batchnorm, Dropout and Convolutions (Coding Question)

In this assignment, you will implement batch normalization, dropout, and convolutions using NumPy and PyTorch. For this assignment, we recommend using Google Colab as some PCs or laptops may not support GPU-based PyTorch.

Attention: This coding task will take approximately 4 to 6 hours to complete, taking into account the time required for training the model. Please plan accordingly and start early to ensure adequate time for completion.

The assignment consists of three parts:

Implementing Batch Norm and Dropout. Open [bn_drop.ipynb](#) and follow the instructions in the notebook. You will implement the forward and backward propagation of dropout and batch normalization in NumPy. A GPU runtime is not required for this part.

Implement convolution and spatial batch norm. Open [cnn.ipynb](#) and follow the instructions in the notebook. You will implement the forward and backward propagation of convolutional layers and spatial dropout in NumPy. A GPU runtime is not required for this part.

Use deep learning framework. Open [pytorch_cnn.ipynb](#) and follow the instructions in the notebook. For this part, you will need to switch to a GPU runtime (details can be found in the notebook). You will implement a convolutional neural network with convolution layers, batch normalization, and dropout using PyTorch and train it on a GPU. You also have the opportunity to improve or design your own neural network.

Please answer the following question in your submission:

- (a) **Draw the computational graph of training-time batch normalization** In input of the computational graph should be $\mathbf{X}, \gamma, \beta$, the output of the computational graph should be $\mathbf{Y}$, and the intermediate nodes are $\mu, \sigma^2, \mathbf{Z}$.
- (b) **(Optional) Derive the closed-form back-propagation of a batch normalization layer (during training).** Include the answer in your written assignment.
Specifically, given $dy_{i,j} = \frac{\partial \mathcal{L}}{\partial Y_{i,j}}$ for every i, j , Please derive $\frac{\partial \mathcal{L}}{\partial X_{i,j}}$ for every i, j as a function of $dy, \mathbf{X}, \mu, \sigma^2, \epsilon, \gamma, \beta$.
- (c) **Explain what you see in this experiment. What does it suggest about dropout?**
- (d) **Briefly describe your neural network design and the procedure of hyperparameter tuning.**

9. Homework Process and Study Group

Citing sources and collaborators are an important part of life, including being a student!

We also want to understand what resources you find helpful and how much time homework is taking, so we can change things in the future if possible.

- (a) **What sources (if any) did you use as you worked through the homework?**
- (b) **If you worked with someone on this homework, who did you work with?**
List names and student ID's. (In case of homework party, you can also just describe the group.)
- (c) **Roughly how many total hours did you work on this homework?**

Contributors:

- Saagar Sanghavi.
- Romil Bhardwaj.
- Peter Wang.
- Jerome Quenum.
- Anant Sahai.
- Linyuan Gong.
- Sheng Shen.
- Suhong Moon.
- Jake Austin.
- Kevin Li.